

STREAMIND: A Real-Time AI Meeting Intelligence Pipeline

Mervegul Parlak, Alperen F. Zengin,
Ayse B. Demir, Zafer Gurel, Ali C. Begen
Ozyegin University
Istanbul, Turkey

Burak Kara
Inria
Rennes, France

Abstract—**TODO:** Abstract: real-time meeting intelligence pipeline; scoring-formula-driven design; key contributions (streaming dedup, keyword guarantee, quality-gated latency optimisation); headline result.

Index Terms—real-time speech processing, automatic speech recognition, large language models, streaming summarisation, WebRTC, Juturna

I. INTRODUCTION

TODO: Motivate live-audio intelligence (meetings, podcasts, IETF sessions). Frame the STREAMIND scoring formula $C_i = B_i + K_i + L_i$ with the $B_i \geq 10$ latency gate as the central design constraint: *quality first, latency second, keyword discipline always*.

Contributions. TODO: Enumerate three contributions: (1) streaming chunk deduplication that preserves temporal coherence across overlapping 5s ASR chunks while keeping the aggregated window non-redundant; (2) a keyword guarantee algorithm that strips banned generic terms and back-fills missing slots from transcript proper nouns, converting the worst-case -6 LLM-failure mode into a $+0$ floor; (3) quality-gated latency optimisation through prompt engineering and 4-bit quantisation, jointly cutting `proc_time` by 24% while keeping B_i at ceiling.

Paper roadmap. TODO: Section ordering pointer.

II. BACKGROUND AND RELATED WORK

TODO: Juturna node-graph framework. Janus WebRTC gateway. Streaming ASR: Whisper family, faster-whisper int8. LLM summarisation under latency constraints; small-model 4-bit quantisation (Qwen3.5 OptiQ). Position STREAMIND as a *systems* contribution wrapping these primitives, not a new model.

III. SYSTEM ARCHITECTURE

TODO: High-level pipeline overview: insert Figure 1. Explain seven-stage data flow from Opus/RTP ingestion to challenge-endpoint POST. State the scoring-aware design principle: every component is justified against one or more terms of $C_i = B_i + K_i + L_i$.

A. Audio Reception

TODO: Janus WebRTC gateway forwards mono Opus/48k as RTP to `audio_rtp` on UDP/8888. Identical code paths in dev and eval environments; earns the flat $+4$ audio-level Janus bonus. Cite Janus.

B. Incremental ASR

TODO: `audio_chunker` buffers the stream into 5s chunks with 1s overlap (`advance = 4s`) so the ASR sees audio incrementally. `transcriber_whisper` runs `faster-whisper small.en (int8, device: auto)`. VAD filter enabled; previous-text conditioning disabled to avoid context drift.

C. Hallucination Filter

TODO: Post-ASR guard. Strips Whisper noise tags (`[Music]`, `(applause)`) and drops pure-hallucination outputs against a configurable exact-match block-list. Protects B_i factual-consistency. Insert Figure 3 here or in §III-D.

D. Novel Chunk Extraction

TODO: Word-level overlap detection: longest suffix-prefix match up to `max_overlap_words=10` with a `SequenceMatcher` fallback (`similarity_threshold=0.8`). Emits only novel words. Preserves temporal coherence across overlapping ASR chunks; prevents duplicate content reaching the aggregator. Refer to Figure 3.

E. Window Aggregation

TODO: `window_aggregator` concatenates novel transcript segments into a rolling 300s context window. Flush trigger: `chunk_end ≥ window_start + 300s`. Graceful stop emits partial windows on pipeline shutdown.

F. Summarisation

TODO: Two interchangeable backends with identical output schema: `summarizer_llm` (Ollama, Qwen3.5-4B, CUDA target) and `summarizer_mlx` (MLX, Qwen3.5 2B/4B/9B OptiQ-4bit, Apple Silicon dev). Prompt engineering for conciseness + specificity (1–2 sentences, specific noun-phrase keywords) double-up: higher B_i conciseness score and lower `num_predict`, which boosts L_i . /no_think

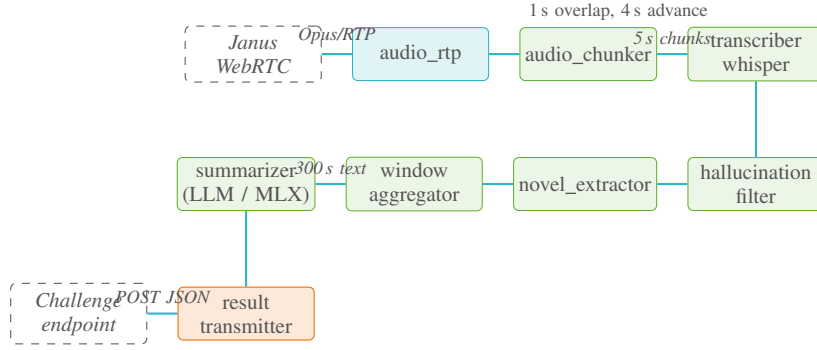


Fig. 1: End-to-end STREAMIND pipeline. Janus delivers mono Opus/RTP into `audio_rtp`; chunks slide every 4 s through ASR; deduplication and hallucination filtering precede a 300 s aggregation buffer; the summariser emits `{summary, keywords[3]}` per window; the sink stores locally and POSTs to the challenge endpoint.

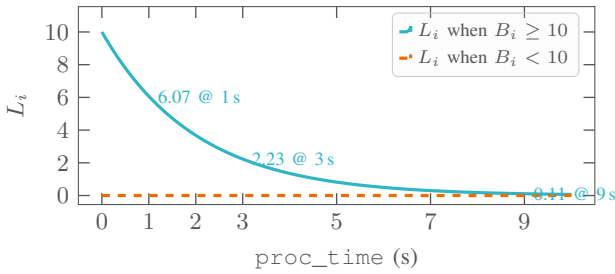


Fig. 2: Latency bonus $L_i = 10e^{-0.5 \cdot \text{proc_time}}$, gated by $B_i \geq 10$. Below the gate, latency contributes zero; above it, returns diminish past $\sim 1\text{--}2$ s.

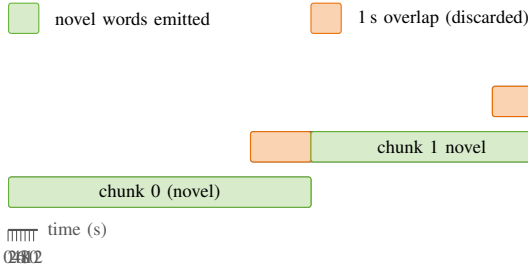


Fig. 3: Audio chunking and novel extraction timing. Five-second chunks slide by 4 s; the novel extractor isolates the 4 s of newly-spoken content each step, dropping the 1 s of overlap with the previous chunk.

flag suppresses chain-of-thought. Robust output cleaning strips ChatML tokens (`<|im_end|>`, `<|endoftext|>`) and extracts the outermost balanced JSON object.

G. Keyword Validation

TODO: Shared ensure_three_keywords (plugins/nodes/proc/_summarizer_common/keywords.py) guarantees exactly three keywords regardless of LLM output state. Pipeline: (i) deduplicate case-insensitively, (ii) drop banned generic terms (“general”, “discussion”, “meeting”), (iii) back-fill from transcript by frequency + capitalisation

TABLE I: Pipeline nodes (Juturna).

TODO: Rows: `audio_rtp`, `audio_chunker`, `transcriber_whisper`, `hallucination_filter`, `novel_extractor`, `window_aggregator`, `summarizer_llm/summarizer_mlx`, `result_transmitter`. Columns: type, model/algorithm, key hyper-parameters, role.

TABLE II: Summariser profiles.

TODO: Columns: profile, backend, model, quantisation, temp, top_p, repetition_penalty, mem footprint, target hardware. Rows: `ollama-qwen3.5-4b` (submission default, CUDA), `ollama-qwen3.5-9b`, `mlx-Qwen3.5-2B-OptiQ-4bit`, `mlx-Qwen3.5-4B-OptiQ-4bit`, `mlx-Qwen3.5-9B-OptiQ-4bit`.

ranking, (iv) pad with conservative fallbacks. Converts -6 worst-case into $+0$ floor.

H. Result Transmission

TODO: `result_transmitter` maps internal keys to challenge contract (`window_start` $\rightarrow$ `from`, `window_end` $\rightarrow$ `to`, `latency` $\rightarrow$ `proc_time`), writes `results/{model}/{window}/window_N.json`, and POSTs to `destination_endpoint` when configured. Drains pending messages on stop.

IV. IMPLEMENTATION AND DEPLOYMENT

TODO: Pipeline-config assembly: `tools/assemble_config.py` merges `config-base.json` with a summariser profile JSON. Launcher: `tools/run_pipeline.sh`.

TODO: Single-image Docker submission: CUDA 12.3 runtime, Python 3.12 + uv, Ollama daemon, pre-pulled Qwen3.5-4B baked at build time. `docker-compose.yml` orchestrates Janus + pipeline. Entry point starts Ollama, waits for readiness, then launches the pipeline with the submission profile.

TODO: Reproducibility checklist: dataset hashes, model revision pins, RTP payload_type, encoding_clock_chan.

TABLE III: Per-window scores on youtube_15min.wav, 300 s windows, Qwen3.5-4B (CUDA).

TODO: Columns: window, from (s), to (s), proc_time (s), B_i , K_i , L_i , C_i . Numbers TBD from CUDA bench.

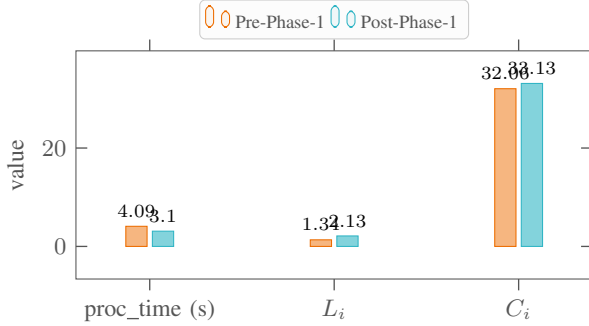


Fig. 4: Phase-1 ablation. Pre-Phase-1 vs Post-Phase-1 proc_time, L_i , and C_i on 30s windows of youtube_15min.wav. Phase-1 fixes (prompt tightening, num_predict=150, banned-keyword filter, JSON extraction fix) cut proc_time 24% and lifted L_i 59% while holding B_i at ceiling.

V. EVALUATION

A. Setup

TODO: Datasets: youtube_15min.wav 120s slice (3 full 300s windows); secondary against the rev16 Whisper subset (30 podcast episodes) and ietf IETF125 sessions. Judge harness: tools/eval_quality.py with local qwen3.5:9b-16k as judge. Disclose: this approximates the official judge; numbers are directional. Hardware: Apple Silicon M-series (MLX, dev); NVIDIA RTX 4090 dev box and RTX Pro 4500 target (CUDA, Ollama).

B. End-to-End Scoring

TODO: Per-window $B_i/K_i/L_i/C_i$ and audio-level mean across summariser profiles. Refer to Table III.

C. Latency Profile

TODO: proc_time distribution; refer to Figure 2 for the L_i curve. Optional per-stage breakdown (ASR vs LLM) if instrumentation lands.

D. Ablations

TODO: Four single-component removals plus the Phase-1 fix package. See Figure 4 and Table IV.

E. Model-Size Sweep

TODO: Qwen3.5 2B vs 4B vs 9B. MLX (Apple Silicon) and CUDA (Ollama, RTX Pro 4500). See Figure 5.

F. Threats to Validity

TODO: Local judge $\neq$ official judge; held-out audio is small; deterministic decoding settings; eval hardware availability.

TABLE IV: Component ablation. Δ relative to the full pipeline.

TODO: Rows: baseline (full pipeline), -banned_keywords, -hallucination_filter, -novel_extractor. Columns: ΔB_i , ΔK_i , ΔL_i , ΔC_i . Numbers TBD from CUDA bench.

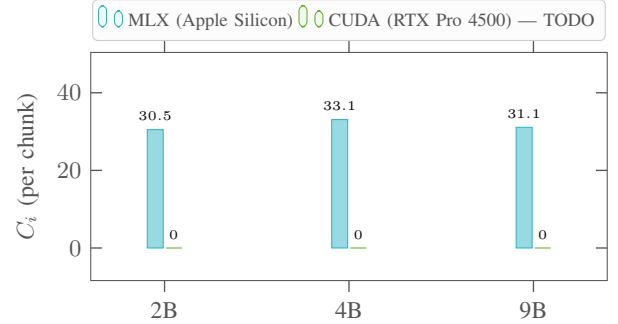


Fig. 5: Quality vs latency across Qwen3.5 model sizes (2B/4B/9B) on MLX (Apple Silicon) and Ollama (RTX Pro 4500). Larger models trade proc_time for marginal B_i gains; the gate at $B_i \geq 10$ makes the sweet spot model-size-dependent.

VI. DISCUSSION

TODO: What the scoring formula encourages vs what real users want (gist coherence over keyword precision; latency only matters once quality clears the gate). Limits of static prompt tuning. Future work: per-window adaptive prompts, speculative decoding, batched ASR, structured-output decoding.

VII. CONCLUSION

TODO: Recap three contributions. State final claimed audio-level score (avg C_i + 4 Janus bonus) on the submission profile. Point at the open-source release.